# DD2424 Assignment 2 Simon Yu

Simon Yu

April 2026

## Analytical Gradient Check

In order to check the validity of the analytic gradient computations the same method as in Assignment 1 where the analytical gradient is compared to the PyTorch gradient that is computed with the given function`ComputeGradsWithTorch`. The analytical and PyTorch gradients are then compared when it comes to shape and their absolute- and relative error. As can be seen in the results the errors are all very small and the gradients hold the same shape thus we can say that our analyrical gradient computation is correct.

```
Comparing W0:
shape my_grads   : (6, 5)
shape torch_grads: (6, 5)
max absolute difference: 4.163336342344337e-17
max relative error     : 3.4309265627951477e-16

Comparing b0:
shape my_grads   : (6, 1)
shape torch_grads: (6, 1)
max absolute difference: 2.0816681711721685e-17
max relative error     : 2.8412300043204553e-16

Comparing W1:
shape my_grads   : (10, 6)
shape torch_grads: (10, 6)
max absolute difference: 3.3306690738754696e-16
max relative error     : 3.049679241003202e-16

Comparing b1:
shape my_grads   : (10, 1)
shape torch_grads: (10, 1)
max absolute difference: 1.1102230246251565e-16
max relative error     : 9.687386046832385e-17
```

# Exercise 3

To test the implementation of cyclic learning rate the network was trained over one cycle which resulted in an test accuracy of 45.81% and the training curves seen in Figure 1

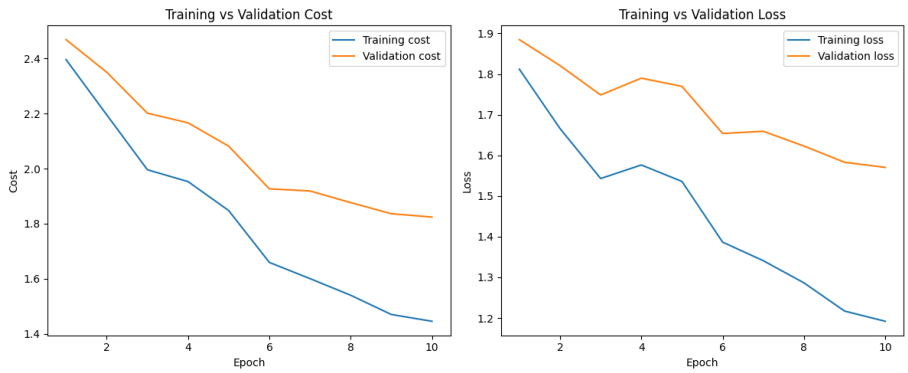


Figure 1: Training curves (cost, loss) for one cycle of training. The hyper-parameter settings of the training algorithm are `eta_min = 1e-5, eta_max = 1e-1, lam=.01 and n_s=500.`

# Exercise 4

After testing the implementation the classifier was trained over three cycles which yielded in a test accuracy of 47.57% and the training curves can be seen in Figure 2

## Coarse to fine random search

For the coarse search `l_min=-5 and l_max=-1` was used and 20 lambdas were randomly generated between $10^{l_min}$ `and` $10^{l_max}$ with the parameter values of `n_batch=100, eta_min=1e-5, eta_max=1e-1, and n_s=2*np.floor(n/n_batch)` and training over 2 cycles, the 3 best performing networks were:

```
lambda = 1.13104e-03, val acc = 0.5288
lambda = 1.71555e-03, val acc = 0.5254
lambda = 4.55876e-05, val acc = 0.5254
```

The fine search was then conducted with `l_min=-3.5 and l_max=-2.5` since this is was the range of the majority of the five best performing lambda values. 20 new values were drawn in the new range in the same way as for the coarse search and the network is now trained over three cycles. The three best validation accuracies were given by:

Figure 2: Training curves (cost, loss) for three cycle of training. The hyper-parameter settings of the training algorithm are `eta_min = 1e-5, eta_max = 1e-1, lam=.01 and n_s=800.` (should say update step not epoch)

```
lambda = 7.25856e-04, val acc = 0.5340
lambda = 5.35377e-04, val acc = 0.5326
lambda = 1.03126e-03, val acc = 0.5322
```

Using `lambda=7.25856e-04` that resulted in the best validation accuracy and also increasing `n_s` to `n_s=4*np.floor(n/n_batch)` and a test accuracy of 51.47% was acheived after training the network over three cycles. The training and validation loss and cost plot can be seen in Figure 3

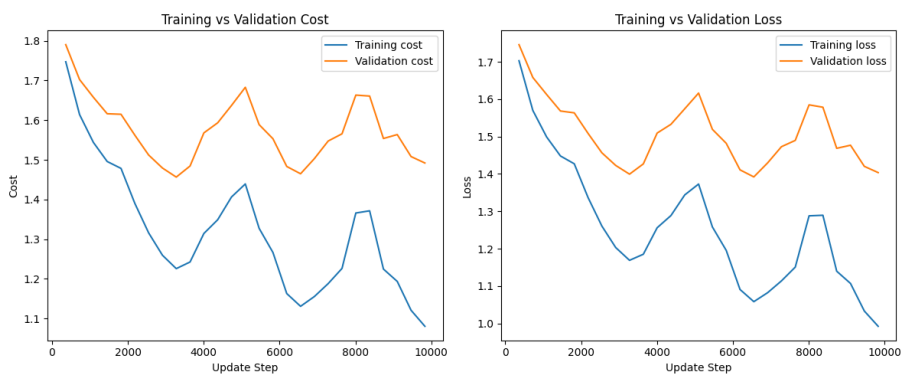


Figure 3: Training curves (cost, loss) for three cycle of training. The hyper-parameter settings of the training algorithm are `eta_min = 1e-5, eta_max = 1e-1, lam=7.25856e-04 and n_s=4*np.floor(n/n_batch)`